Desenvolvimento de Sistemas Web II

HTTP na prática, Cache, REST e Clean Architecture

DIM0547 — Turma 01 · Aula 02 (Sprint 0)

Prof. Fernando · UFRN · 2026.2

Onde paramos

Na aula passada vimos a **superfície** do sistema:

```
Recurso · identificador · representação
Mensagem HTTP: linha inicial, cabeçalhos, corpo
Métodos: seguro e idempotente
Status: as cinco famílias e os pares que confundem
Erro em application/problem+json
```

Faltaram duas peças que fecham esse assunto: **cache** e **REST**.

Antes delas, uma recapitulação rápida com o DevTools aberto — para que todo mundo esteja no mesmo ponto.

Onde estudar depois

Guia	Foco
MDN — HTTP · `developer.mozilla.org/docs/Web/HTTP`	Referência da web
RFC 9110 — HTTP Semantics	A especificação, quando a dúvida for fina
RFC 9111 — HTTP Caching	O assunto da Parte 2

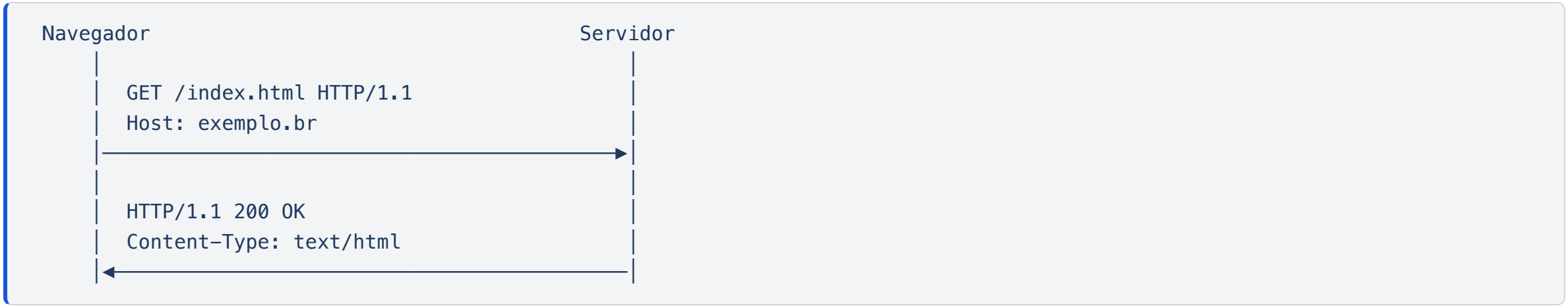
O MDN é a referência que vocês vão consultar o semestre inteiro.

Parte 1

HTTP na prática

Abram o Chrome e o DevTools com `F12`.

O que acontece ao abrir um site



Uma página comum dispara dezenas dessas trocas: HTML, CSS, imagens, fontes, chamadas de API.

DevTools: a aba Network

`F12` → **Network** → recarregue a página.

Coluna	O que mostra
Name	O recurso pedido
Status	O código de resposta
Type	`document`, `script`, `fetch`, `xhr`…
Size	Tamanho, e se veio do cache
Time	Duração da requisição

O filtro **Fetch/XHR** isola as chamadas de API e esconde imagens e scripts.

Cabeçalhos que importam

Cabeçalho	Onde	Para quê
`Content-Type`	ambos	Formato do corpo
`Accept`	requisição	Formato que o cliente prefere
`Authorization`	requisição	Credencial
`User-Agent`	requisição	Quem está chamando
`Cache-Control` · `ETag`	ambos	Cache e revalidação
`Location`	resposta	Onde o recurso criado ficou
`Retry-After`	resposta	Quando tentar de novo

As três últimas linhas são o assunto das duas próximas partes.

Parte 2

Cache

O custo de uma requisição

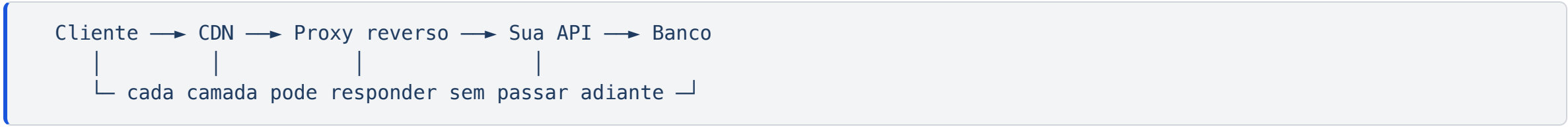
Um `GET /produtos` que leva 200 ms envolve, no mínimo:

```
resolução de DNS  +  handshake TCP  +  handshake TLS
+  roteamento  +  consulta ao banco  +  serialização
```

Otimizar a consulta ao banco melhora uma fatia. **Não fazer a requisição** elimina todas.

> Cache não é otimização de última hora: é uma decisão de projeto da API, e aparece nos cabeçalhos que você escolhe devolver.

Onde o cache mora

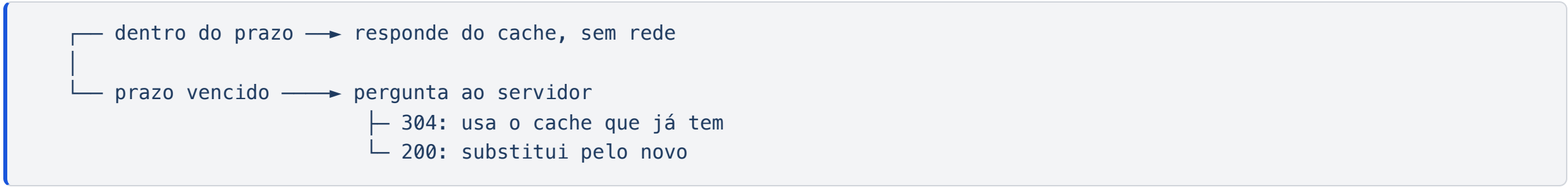


Camada	Guarda o quê	Controlada por
Cliente	Respostas das próprias requisições	Cabeçalhos da resposta
CDN e proxy	Respostas compartilhadas entre usuários	Cabeçalhos da resposta
Aplicação	Resultados de consulta, objetos	Você, no código

> As duas primeiras camadas você controla **sem escrever código**, apenas declarando cabeçalhos corretos.

Dois mecanismos, não um

Mecanismo	Pergunta	Custo quando acerta
Frescor	Ainda vale?	Zero: ninguém fala com o servidor
Validação	Mudou desde a última vez?	Uma ida e volta, sem corpo



Os dois se combinam: frescor curto com validação é a configuração mais comum em API, porque mantém o dado atual sem pagar o corpo da resposta toda vez.

Frescor: `Cache-Control`

```
Cache-Control: public, max-age=300, stale-while-revalidate=60
```

Diretiva	O que significa
`public`	Qualquer cache pode guardar, inclusive CDN e proxy
`private`	Só o cache do cliente. Use quando a resposta depende do usuário
`max-age=300`	Válido por 300 segundos a partir de agora
`no-cache`	Pode guardar, mas **precisa revalidar** antes de usar
`no-store`	Não pode guardar em lugar nenhum
`stale-while-revalidate=60`	Serve o antigo e busca o novo em segundo plano

> `no-cache` e `no-store` são confundidos o tempo todo. `no-cache` guarda e pergunta; `no-store` não guarda.

Validação: `ETag`

O servidor devolve uma etiqueta que identifica **aquela versão** da representação:

```
GET /produtos/42                    ⟶    HTTP/1.1 200 OK
                                         ETag: "a1b2c3"
                                         Cache-Control: max-age=60
```

Depois que o prazo vence, o cliente pergunta em vez de baixar de novo:

```
GET /produtos/42                    ⟶    HTTP/1.1 304 Not Modified
If-None-Match: "a1b2c3"                  ETag: "a1b2c3"
                                         (sem corpo)
```

> `304` é a resposta mais barata do protocolo: confirma que o dado está atual sem transmitir o dado.

Forte, fraca e a alternativa por data

Forma	Exemplo	Quando usar
ETag forte	`"a1b2c3"`	Byte a byte idêntico. Padrão para JSON
ETag fraca	`W/"a1b2c3"`	Equivalente o bastante, embora não idêntica
Última modificação	`Last-Modified` / `If-Modified-Since`	Alternativa mais simples, com resolução de 1 segundo

Como gerar a ETag: hash do corpo, número de versão da linha no banco, ou o `updated_at` da entidade.

> Resolução de um segundo é uma limitação real: dois updates no mesmo segundo ficam indistinguíveis. Em API, prefira ETag.

Parte 3

REST

O que REST é

REST é um **estilo arquitetural**, descrito por Roy Fielding em 2000, na tese em que ele explicava por que a web escalou.

O nome não menciona JSON, nem HTTP, nem CRUD. Descreve um conjunto de **restrições** — coisas que você abre mão de fazer, em troca de propriedades que ganha.

> A confusão vem daí: "API REST" virou sinônimo de "API que devolve JSON sobre HTTP". A maioria dessas APIs atende parte das restrições, e isso é legítimo — desde que a equipe saiba qual deixou de fora, e por quê.

As restrições, e o que cada uma compra

Restrição	O que exige	O que você ganha
Cliente-servidor	Separar interface de armazenamento	Evoluir os dois lados em ritmos diferentes
Sem estado	Cada requisição se basta	Qualquer réplica atende qualquer requisição
Cacheável	A resposta declara se pode ser guardada	O que vimos na parte anterior
Interface uniforme	Recursos identificados, manipulados por representação	Cliente genérico funciona com qualquer serviço
Em camadas	O cliente não sabe se fala com a origem	Proxy, CDN e balanceador entram sem mudar o cliente
Código sob demanda	Servidor pode enviar código executável	Opcional, e quase nunca usada

> Repare que **cacheável** é uma das restrições. Cache não é acessório de REST: é parte da definição.

Modelo de maturidade de Richardson

Nível	O que faz	Exemplo
0	Uma URI, um verbo, tudo no corpo	`POST /api` com `{"op": "criarPedido"}`
1	Recursos identificados por URI	`POST /pedidos/criar`
2	Verbos e status com semântica	`POST /pedidos` → `201` + `Location`
3	A resposta traz os próximos passos	corpo com links `cancelar`, `pagar`

O nível 2 é o alvo do curso, e é o que a rúbrica cobra a partir da Sprint 1.

> O nível 3, chamado HATEOAS, é pouco adotado na prática. Vale conhecer para saber o que se está deixando de fora quando se diz "nossa API é REST".

Nível 2, na prática

```
POST /pedidos                  HTTP/1.1 201 Created
Content-Type: application/json Location: /pedidos/9f2c
                         ⟶     ETag: "v1"

GET /pedidos/9f2c              HTTP/1.1 200 OK
                         ⟶     Cache-Control: private, max-age=30

DELETE /pedidos/9f2c           HTTP/1.1 204 No Content
If-Match: "v1"           ⟶
```

Três operações, três métodos com semântica, três status corretos, cabeçalhos coerentes.

Checklist de uma API nível 2

- [ ] URIs nomeiam **recursos**, com substantivos no plural, sem verbos
- [ ] `GET` nunca altera estado
- [ ] `POST` cria e devolve `201` com `Location`
- [ ] `PUT` e `DELETE` são idempotentes
- [ ] Erro usa a família de status correta, com corpo em `application/problem+json`
- [ ] `GET` declara `Cache-Control` e `ETag`
- [ ] Escrita concorrente é protegida por `If-Match`

> São os sete itens que a rúbrica da Sprint 1 verifica. Todos podem ser conferidos com `curl -v` antes de entregar.

Parte 4

Projetando a sua API

Recurso, não ação

A URL nomeia **coisas**; o método diz o que fazer com elas.

Evitar	Preferir
`POST /criarPedido`	`POST /pedidos`
`GET /buscarPedidoPorId?id=7`	`GET /pedidos/7`
`POST /pedidos/7/deletar`	`DELETE /pedidos/7`
`GET /listarPedidosDoCliente?c=3`	`GET /clientes/3/pedidos`

Substantivos no plural, hierarquia refletindo a relação entre entidades.

O ganho não é estético: um cliente que entende `/pedidos/7` consegue prever `/clientes/3` sem ler documentação.

Escolher o status certo

Situação	Status
Leitura bem-sucedida	`200 OK`
Recurso criado	`201 Created` + cabeçalho `Location`
Remoção bem-sucedida, sem corpo	`204 No Content`
JSON malformado	`400 Bad Request`
Sem credencial	`401 Unauthorized`
Credencial válida, sem permissão	`403 Forbidden`
Recurso inexistente	`404 Not Found`
Precondição `If-Match` falhou	`412 Precondition Failed`
JSON válido, regra de negócio violada	`422 Unprocessable Content`

A diferença entre `400` e `422` costuma ser cobrada em revisão de código: o primeiro é sintaxe, o segundo é semântica.

Erros que dá para ler

Um `400` sem corpo obriga quem consome a adivinhar. A RFC 9457 padroniza o formato:

```
HTTP/1.1 422 Unprocessable Content
Content-Type: application/problem+json

{
  "type": "https://exemplo.br/erros/pedido-sem-itens",
  "title": "Pedido precisa de ao menos um item",
  "status": 422,
  "detail": "O pedido enviado tem a lista de itens vazia.",
  "instance": "/pedidos"
}
```

O `type` é uma URI que identifica a **classe** do erro; o `detail` descreve **aquela** ocorrência.

Filtro, ordenação e paginação

```
GET /pedidos?status=aberto&criado_ate=2026-08-01&ordenar=data&limite=20&pagina=2
```

Parâmetro	Papel
`status`, `criado_ate`	Filtro
`ordenar`	Ordenação, declarada por quem consulta
`limite`, `pagina`	Paginação

Duas estratégias de paginação

Estratégia	Como funciona	Problema
`limit` / `offset`	Pula N, traz M	Itens deslocam se a coleção muda
Cursor	"continue a partir daqui"	Não permite pular para a página 7

A resposta precisa dizer onde o cliente está: total de itens, página atual, ou um link para a próxima.

Sem paginação, uma consulta ampla devolveria dezenas de milhares de registros.

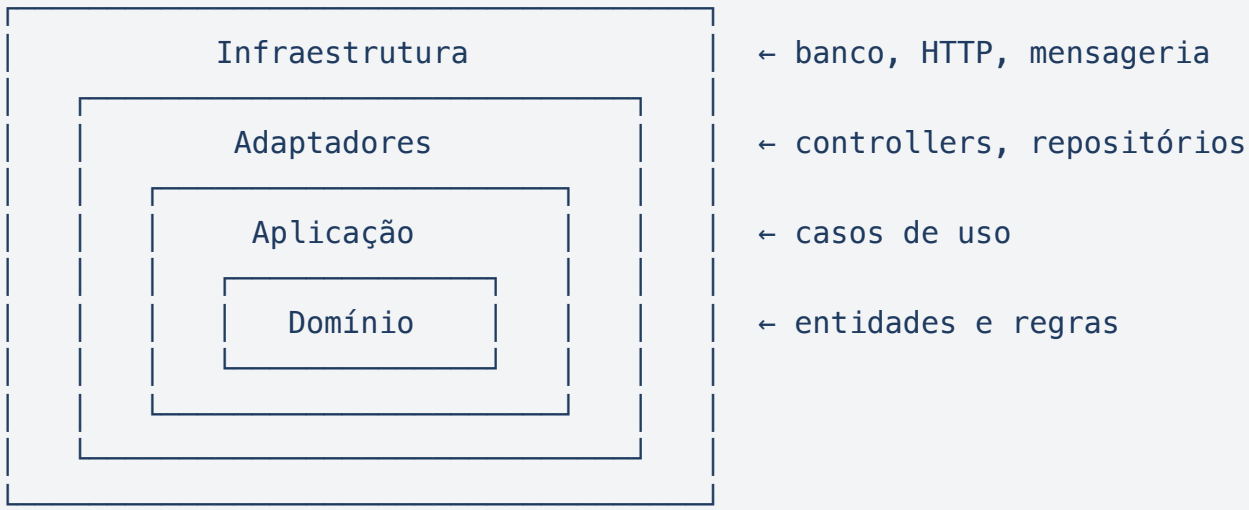
Ferramentas úteis

Ferramenta	Para quê
DevTools (`F12`) → aba Network	Ver o que a **página** pediu, e o que veio do cache
Hoppscotch · `hoppscotch.io`	Fazer **você** a requisição, com os cabeçalhos que quiser

O Hoppscotch roda no navegador, sem cadastro, e faz o mesmo papel do Postman.

> Os conceitos das partes anteriores não são teóricos: todos aparecem em cabeçalhos que dá para ler agora.

A regra de dependência



O que mora em cada camada

Camada	Contém	Nunca contém
Domínio	Entidades, objetos de valor, regras invariantes	Anotação de framework, SQL, HTTP
Aplicação	Casos de uso, orquestração, **interfaces** de repositório	Detalhe de persistência ou de transporte
Adaptadores	Controllers, implementações de repositório, mapeadores	Regra de negócio
Infraestrutura	Configuração, cliente de banco, cliente HTTP	Regra de negócio

Formação dos grupos

- **1 a 4 integrantes.** Cinco só com justificativa aprovada
- Formação até a entrega da Sprint 0
- Alterações valem a partir da sprint seguinte

Registrem no `README.md` do repositório: nome do grupo, integrantes com matrícula, e o usuário GitHub de cada um.

> Quem ainda não tem grupo: usem o canal do Discord.

MUSI

O que a entrega pede	Onde ver no MUSI
Visão do produto, no template	`docs/proposta.md` §1
MVP: dentro e fora, com hipótese de valor	`docs/proposta.md` §2
Backlog priorizado e estimado	`processo/backlog.yaml` — 13 itens
Divisão Kotlin × Go, com justificativa	`docs/proposta.md` §4.1
Entidades principais do domínio	`docs/proposta.md` §5 e `shared/`
Equipe, coorte, integração	`docs/proposta.md` §6 e §7
Monorepo com CI verde	`.github/workflows/ci.yml`

Para quarta, 26/08 — perguntas a trazer

O encontro é **online, no horário da aula**, vocês podem até preparar um esboço do que deve ser entregue e mostrar pra mim, eu vou dar feedback ao vivo.

Próximos passos

Tarefas

- Fechar o grupo e registrar no `README.md` do repositório
- Escrever a visão do produto e o recorte do domínio, usando o template de `docs/SPRINT-0.md`
- Rascunhar a divisão entre o serviço Java e o serviço Go
- Escolher uma API pública e explorá-la no Hoppscotch
- Dar uma olhada no MUSI: `github.com/fmarquesfilho/musi`

> Quarta é encontro **online**, no horário da aula, para dúvidas sobre a proposta e o domínio.

Referências da aula

Protocolo, cache e REST

- MDN — *HTTP* e *HTTP caching* · `developer.mozilla.org/docs/Web/HTTP`
- RFC 9110 — *HTTP Semantics* · RFC 9111 — *Caching* · RFC 9457 — *Problem Details*
- Fielding (2000) — *Architectural Styles*, cap. 5
- Fowler — *Richardson Maturity Model* · `martinfowler.com`

Ktor e implantação

- Ktor · `ktor.io/docs` · Assistente de projeto · `start.ktor.io`
- Koin · `insert-koin.io/docs`
- Render · `render.com/docs` · Neon · `neon.tech/docs`

Disciplina

- `github.com/fmarquesfilho/web2-2026-2`
- Projeto de exemplo · `github.com/fmarquesfilho/musi`